

Seminario de Lenguajes opción .NET

1º Semestre - 2026

Importante

Queremos recordarles la importancia de este primer trabajo del curso. Aunque no recibirá una calificación numérica, será objeto de corrección y dará lugar a una devolución que servirá como guía para continuar hacia el próximo trabajo.

La entrega de este primer trabajo es de carácter obligatorio y requisito excluyente para la aprobación del curso.

La entrega debe ser significativa. No se admitirán entregas en blanco ni aquellas que muestren un esfuerzo mínimo, en lugar de un sincero compromiso y dedicación en la realización del trabajo.

Entrega

Fecha límite: **19/05/2026 hasta las 13:00 hs.**

El trabajo puede realizarse en grupo de hasta 4 integrantes.

La entrega se realizará por medio de un formulario de Google que se publicará más adelante.

Se deberá entregar:

- El código de la solución completa
 - Un documento explicativo (puede ser **.pdf** o **.md**) donde se detalle cómo probar la funcionalidad desarrollada desde Program.cs. Se debe explicitar código de ejemplo junto con la salida por consola producida.
 - Subir la solución completa y el documento explicativo, todo comprimido en un único archivo (preferentemente **.zip**). El nombre del archivo zip debe contener los apellidos de los autores del trabajo, ejemplo: *Apellido1_Apellido2_Trabajo1.zip*.
-

Trabajo Práctico: Sistema para la Gestión de Expedientes (SGE)

Fase 1: Dominio Rico y Arquitectura Limpia

(La Fase 2 será la entrega final)

Descripción General

El objetivo de este trabajo práctico es desarrollar el núcleo lógico de un Sistema de Gestión de Expedientes (SGE). Este sistema posibilita la organización y administración efectiva de la información, donde cada expediente contendrá un conjunto de datos propios junto con una colección de trámites asociados.

Esta primera fase se construirá utilizando los principios de **Arquitectura Limpia** y tácticas de **Domain-Driven Design (DDD)** vistas en la teoría. El objetivo es lograr un código altamente desacoplado, testeable y preparado para evolucionar. En una entrega futura este mismo núcleo de negocio se conectará a una base de datos SQLite y a una interfaz web (Blazor o Web API) sin necesidad de modificar las reglas de negocio.

1. Estructura de la Solución

La solución deberá llamarse **SGE** y deberá contener estrictamente los siguientes cuatro proyectos, respetando la Regla de Dependencia:

1. **SGE.Dominio** (Class Library - No depende de nadie)
2. **SGE.Aplicacion** (Class Library - Depende de SGE.Dominio)
3. **SGE.Infraestructura** (Class Library - Depende de SGE.Aplicacion y SGE.Dominio)
4. **SGE.Consola** (Console App - Depende de todos para actuar como *Composition Root*, es decir, el punto central donde se crean las instancias de las clases proveyéndoles las dependencias necesarias)

1.1 Organización Interna (Feature Folders)

Dentro de cada proyecto (especialmente en Dominio y Aplicación), se recomienda adoptar la organización por **Módulos de Negocio (Feature Folders)** en lugar de la clásica agrupación por "tipo técnico" (se debe evitar crear carpetas genéricas llamadas **Entidades**, **DTOs** o **CasosDeUso**).

Para los nombres de estas carpetas utilizaremos el **plural** (**Expedientes**, **Tramites**), mientras que las clases en su interior irán en **singular** (**Expediente**, **Tramite**).



Nota Técnica (C#): ¿Por qué la carpeta en plural y la clase en singular? Microsoft aconseja firmemente que un espacio de nombres (*namespace*, que se genera a partir del nombre de la carpeta) no se llame exactamente igual que un tipo o clase en su interior. Si la carpeta y la clase se llamaran ambas **Tramite**, el compilador de C# se confundiría al intentar resolver las referencias y generaría errores. Al usar el plural para la carpeta, evitamos esta colisión.



Nota sobre la Convención de Nomenclatura: Notarán a lo largo del TP una mezcla

intencional de idiomas. La convención que adoptamos para la realización de este trabajo es: **Dominio en Español y Roles Técnicos en Inglés**. Utilizamos el español para reflejar el Lenguaje Ubicuo del negocio y de la acción que estamos resolviendo (**Expediente, Tramite, Caratula, Permiso, Autorizacion**). Sin embargo, utilizamos sufijos en inglés para los roles técnicos y patrones de diseño (**UseCase, Repository, Request, Response, Service, Exception**), ya que es el estándar de facto en la industria del software y facilita enormemente la identificación del propósito técnico de cada clase. Ejemplos: **EntidadNoEncontradaException, ExpedienteRepository**.

Ejemplo de la estructura sugerida:

- **SGE.Dominio**
 - 📁 **Expedientes/** (Contendrá la entidad *Expediente, Caratula, EstadoExpediente*)
 - 📁 **Tramites/** (Contendrá la entidad *Tramite, ContenidoTramite, EtiquetaTramite*)
 - 📁 **Comun/** (Contendrá la clase transversal *DominioException*)
- **SGE.Aplicacion**
 - 📁 **Expedientes/** (Casos de Uso de Expedientes, *ExpedienteDTOs, IExpedienteRepository*)
 - 📁 **Tramites/** (Casos de Uso de Trámites, *TramiteDTOs, ITramiteRepository*)
 - 📁 **Autorizacion/** (*IAutorizacionService, Permiso, AutorizacionException*)

💡 **Nota sobre Libertad de Diseño y Excepciones:** A lo largo de este documento especificamos las entidades, interfaces y excepciones mínimas y obligatorias requeridas para el sistema (como **DominioException** o **AutorizacionException**). Sin embargo, se pueden definir nuevas excepciones personalizadas (por ejemplo, una **EntidadNoEncontradaException** para indicar que un ID no fue encontrado al ejecutar un Caso de Uso), clases auxiliares o enumerativos adicionales que se consideren convenientes.

2. Capa de Dominio (SGE.Dominio)

En este proyecto residirá el corazón del software. Aquí definiremos las **Entidades, Objetos de Valor (Value Objects), Enumerativos** y la **DominioException**. No debe existir código de acceso a datos ni referencias a librerías externas.

Entidades y Reglas de Negocio

Se trabajará con dos entidades fundamentales: **Expediente** y **Tramite**. Ambas entidades deben ser autónomas (generar su propio **Guid**), tener constructores que validen su estado inicial, constructores *Factory Method* para su reconstrucción (que utilizará la capa de infraestructura cuando deba reconstruir una entidad a partir de datos almacenados previamente) y propiedades con encapsulamiento (**private set**).

Entidad: Expediente

- **Id** (Guid)
- **Caratula** (Value Object - Encapsulará un dato de texto o *string*)
- **FechaCreacion** (DateTime - Refleja el momento en que se crea la entidad)
- **FechaUltimaModificacion** (DateTime - Refleja el momento en que se actualiza algún dato de la entidad)
- **UsuarioUltimoCambio** (Guid - Id del usuario - Es el identificador del usuario responsable del último cambio)
- **Estado** (Enumerativo: *RecienIniciado, ParaResolver, ConResolucion, EnNotificacion, Finalizado*)
- **Relación con Trámites (Aclaración de Diseño):** Para mantener un bajo acoplamiento, la entidad **Expediente** no debe contener una colección interna de sus trámites (es decir, NO se debe agregar una propiedad como `List<Tramite>`). El Expediente y sus Trámites se persisten y consultan por separado. Será responsabilidad de la capa de Aplicación buscar los trámites correspondientes y pasárselos a la entidad únicamente la información necesaria para aplicar sus reglas.
- **Invariantes de la Entidad:** Todos sus atributos son obligatorios. Al crearse un nuevo expediente de forma natural, su estado inicial debe ser siempre **RecienIniciado**. Además, la **FechaUltimaModificacion** no puede ser menor que la **FechaCreacion**.

Entidad: Trámite

- **Id** (Guid)
- **ExpedienteId** (Guid - Para asociarlo a un expediente)
- **Etiqueta** (Enumerativo: *EscritoPresentado, PaseAEstudio, Despacho, Resolucion, Notificacion, PaseAlArchivo*)
- **Contenido** (Value Object - Encapsulará un dato de texto o *string*)
- **FechaCreacion** (DateTime - Refleja el momento en que se crea la entidad)
- **FechaUltimaModificacion** (DateTime - Refleja el momento en que se actualiza algún dato de la entidad)
- **UsuarioUltimoCambio** (Guid - Id del usuario - Es el identificador del usuario responsable del último cambio)
- **Invariantes de la Entidad:** Todos sus atributos son obligatorios. Además, la **FechaUltimaModificacion** no puede ser menor que la **FechaCreacion**.

Directiva de Diseño: Comportamiento Rico (Métodos de las Entidades) Dado que todas las propiedades utilizan `private set` (encapsulamiento estricto), los Casos de Uso no pueden modificar los datos directamente. Se deberán definir los métodos públicos necesarios en ambas entidades para soportar las operaciones del sistema, garantizando que siempre se respeten las invariantes (por ejemplo, realizando las validaciones adecuadas antes de aceptar un cambio).

Objetos de Valor (Value Objects)

Implementar mediante `record class` al menos los siguientes dos Value Objects para evitar la *Obsesión*

por Primitivos:

1. **Caratula:** Debe validar al instanciarse que el texto no esté vacío ni sea nulo.
2. **ContenidoTramite:** Debe validar al instanciarse que el contenido del trámite (un dato textual) no esté vacío.

Nota de Validación: Si falla la validación al instanciar cualquiera de estos dos Value Objects, o si no se cumple alguna de las invariantes de las Entidades descritas anteriormente, el sistema debe lanzar una **DominioException**.

Lógica de Negocio: Gestión del Estado y Modificación del Expediente

El ciclo de vida de un expediente implica distintos tipos de alteraciones. Para respetar el diseño orientado al dominio, la entidad **Expediente** debe proveer métodos que reflejen la **intención** real del negocio.

A. Modificar Carátula: Si el usuario cometió un error al ingresar el expediente, debe poder corregir la carátula. Esta es una operación excepcional que no afecta el flujo de vida del expediente.

- *Directiva:* La entidad **Expediente** debe exponer un método (ej: `public void ModificarCaratula(Caratula nuevaCaratula, Guid idUsuario)`) que actualice este dato, el usuario responsable y la fecha de modificación.

B. Cambio de Estado Automático (Por trámites): Al agregar, modificar o eliminar un trámite, el sistema puede cambiar automáticamente el estado del expediente según la etiqueta del trámite que haya quedado como "último" (considerando como "último" a aquel trámite con la fecha de creación más reciente):

- Si la etiqueta es **"Resolución"** -> El estado cambia a **"Con resolución"**.
- Si la etiqueta es **"Pase a estudio"** -> El estado cambia a **"Para resolver"**.
- Si la etiqueta es **"Pase al Archivo"** -> El estado cambia a **"Finalizado"**.
- Si el expediente **se queda sin trámites** (ej: se eliminó el único que tenía) -> El estado vuelve a **"RecienIniciado"**.
- *Directiva:* La entidad **Expediente** debe ser la única responsable de evaluar y aplicar esta regla para mutar su estado. Para lograrlo, **sugerimos** que exponga un método (ej: `public bool ActualizarEstado(EtiquetaTramite? ultimaEtiqueta, Guid idUsuario)`) que efectúe el cambio. Sugerimos que retorne **true** si el estado efectivamente cambió (**esta información será muy útil para la orquestación en la capa de Aplicación, que utilizará este valor para saber si debe ordenar al repositorio que persista la modificación, optimizando así las llamadas a la base de datos**) y que reciba el **idUsuario** porque al mutar la entidad, se debe registrar quién causó esta actualización automática. (*Nota de Diseño: Esta estrategia específica en la firma del método —retornar un booleano para optimizar la persistencia, o utilizar un **ultimaEtiqueta** en nulo para representar expedientes que se han quedado sin trámites— es una sugerencia para guiar su diseño. Tienen total libertad para resolver este escenario con otros métodos o parámetros, siempre y cuando*

*respeten la directiva principal: usar un **Modelo Rico** donde la propia entidad concentre la inteligencia y decida sobre su estado. Al recibir solo la etiqueta en lugar del objeto trámite completo, mantenemos las entidades desacopladas).*

C. Cambio de Estado Manual (Flujo natural): El usuario debe poder cambiar el estado de un expediente manualmente, **independientemente de cuál sea su último trámite.**

- *Ejemplo de negocio:* Si el último trámite es una "Resolución", el sistema lo pasa automáticamente a "Con resolución". Sin embargo, más adelante, el usuario debe poder editar el expediente para pasarlo al estado "En notificación" sin necesidad de agregar ningún trámite.
- *Directiva:* La entidad **Expediente** debe exponer un método (ej: `public void CambiarEstado(EstadoExpediente nuevoEstado, Guid idUsuario)`) que reciba el nuevo estado y actualice la información correspondiente.

3. Capa de Aplicación (SGE.Aplicacion)

Esta capa orquesta el sistema. Aquí definiremos los **Casos de Uso**, los **DTOs**, las interfaces y el control de autorizaciones. Las entidades del Dominio no deben "escapar" de esta capa.

Interfaces (Contratos)

Definir las siguientes abstracciones (recomendamos utilizar idioma español para respetar el Lenguaje Ubicuo de nuestro dominio, ej: **ObtenerPorId** en lugar de **GetById**):

- **IExpedienteRepository**
- **ITramiteRepository**
- **IAutorizacionService** (Contendrá el método `bool PoseeElPermiso(Guid idUsuario, Permiso permiso)`)



Consejo de Diseño (Firmas de los Repositorios): Sugerimos que los repositorios contengan métodos base como **Agregar**, **ObtenerPorId**, **Modificar** y **Eliminar**, además de cualquier otro método de consulta que se consideren necesario para sus Casos de Uso (ej. **ObtenerTodos** para expedientes u **ObtenerPorExpedienteId** para trámites). Las operaciones de mutación (**Agregar**, **Modificar**, **Eliminar**) **deben retornar void**. En Arquitectura Limpia, si el repositorio no encuentra el elemento para modificar o eliminar, no debe retornar un booleano **false**, sino lanzar una excepción (ej. **RepositorioException**), ya que intentar operar sobre algo que no existe es un comportamiento anómalo. Por el contrario, los métodos de consulta como **ObtenerPorId** deben estar diseñados para retornar la entidad o **null** (ej: **Expediente?**) en caso de no hallarla. Esta no es una decisión arbitraria: muchas veces un Caso de Uso necesita consultar si una entidad existe (por ejemplo, para evitar crear un duplicado) y actuar en consecuencia. Si el repositorio lanzara una excepción al no encontrarla, nos obligaría a programar el flujo de la aplicación usando bloques **try/catch**, lo

cual es una mala práctica conocida como *Exception Driven Development* (Desarrollo Guiado por Excepciones). Será responsabilidad del Caso de Uso evaluar ese `null` y decidir si debe lanzar una excepción de negocio (ej. `EntidadNoEncontradaException`).

 **Consejo de Diseño (Tipos de Retorno):** Al definir los métodos de los repositorios que devuelven colecciones de objetos (ej. "Obtener todos los expedientes"), es común usar `List<Expediente>`. Sin embargo, para diseñar interfaces verdaderamente resilientes y preparadas para la Fase 2 (cuando nos conectemos a bases de datos), los animamos a **leer el Apéndice A** al final de este documento, donde se explica por qué es preferible usar `IEnumerable<T>`.

Casos de Uso y DTOs

Cada Caso de Uso será una clase independiente (utilizando el sufijo `UseCase`, ej. `AgregarExpedienteUseCase`) con un método `Ejecutar`. La comunicación de entrada y salida se hará estrictamente mediante records DTOs, utilizando los sufijos `Request` y `Response` (ej. `AgregarExpedienteResponse Ejecutar(AgregarExpedienteRequest request)`). Devolver un DTO en lugar de datos primitivos (como un `Guid`) permite que la firma del método sea escalable a futuro.

Se deben implementar, como mínimo, los siguientes Casos de Uso:

1. Alta, Baja y Modificación de Expedientes.

- *Aclaración sobre la Modificación:* Como la entidad separa claramente la intención de *corregir la carátula* (una operación excepcional) y *cambiar el estado manual* (parte del ciclo de vida ordinario), se deben crear dos Casos de Uso totalmente separados: `ModificarCaratulaExpedienteUseCase` y `CambiarEstadoExpedienteUseCase`.
- *Aclaración sobre la Baja (Eliminación en Cascada):* Como regla fundamental de negocio, **al eliminar un expediente, se deben eliminar todos sus trámites asociados**. El Caso de Uso encargado de la baja deberá coordinar esta acción, asegurándose de eliminar **primero** los trámites desde su repositorio, y **luego** el expediente desde el suyo (este orden es vital para evitar errores de integridad referencial a futuro).

2. Alta, Baja y Modificación de Trámites.

- *Aclaración sobre el Cambio de Estado Automático:* Como estas tres operaciones pueden generar la necesidad de actualizar el estado del expediente (alterando cuál es el "último" trámite o cambiando la etiqueta del último trámite), la lógica de orquestación para actualizar el estado es idéntica en los tres. Para evitar repetir código (Principio DRY "**Don't Repeat Yourself**"), se deberá crear un **Servicio de Aplicación** (ej. `ActualizacionEstadoExpedienteService`) e inyectarlo en estos tres Casos de Uso.
- **La responsabilidad del Servicio:** Este servicio actuará como orquestador. Recuperará el expediente, buscará todos sus trámites para determinar cuál es el "último", extraerá la etiqueta de dicho trámite y le pedirá a la entidad que evalúe su nuevo estado invocando:

```
bool cambio = expediente.ActualizarEstado(ultimaEtiqueta, idUsuario).
```

El servicio solo llamará a `_expedienteRepository.Modificar(expediente)` si el método retornó `true`.

 **Nota de Diseño: El poder del Modelo Rico** Noten que el Servicio de Aplicación **no** decide cuál es el nuevo estado ni modifica la propiedad directamente. Toda la lógica sigue residiendo dentro de la propia entidad `Expediente`.

- **Encapsulamiento y Seguridad:** Al usar `{ get; private set; }` en la propiedad `Estado`, impedimos que cualquier componente del sistema cambie el estado accidentalmente saltándose las reglas. El compilador nos obliga a pasar siempre por el embudo de validación.
- **Testeabilidad:** Probar que las reglas de cambio de estado funcionan requiere apenas escribir un Test Unitario rapidísimo instanciando un `Expediente` y llamando a su método. No hace falta crear *mocks* de repositorios.

3. **Listar todos los Expedientes.**

4. **Listar Trámites por Expediente** (Dado un `ExpedienteId`).

Autorización de Usuarios

Definir el enumerativo `Permiso` (`ExpedienteAlta`, `ExpedienteBaja`, `ExpedienteModificacion`, `TramiteAlta`, `TramiteBaja`, `TramiteModificacion`). Todos los `Request` DTOs que impliquen mutación (`Alta`, `Baja`, `Modificación`) deberán recibir el `IdUsuario` (`Guid`) que intenta realizar la acción. El Caso de Uso deberá invocar a `IAutorizacionService.PoseeElPermiso(idUsuario, permisoRequerido)`. Si retorna falso, debe lanzar una `AutorizacionException`.

4. Capa de Infraestructura (SGE.Infraestructura)

Aquí se implementarán los repositorios provisorios y los servicios externos.

- Implementar `ExpedienteTxtRepository` y `TramiteTxtRepository` que guarden los datos en archivos de texto plano.

 **Nota sobre el Rendimiento:** Para las operaciones de **Modificación** y **Baja**, sugerimos el enfoque más sencillo: leer el archivo completo hacia una lista en memoria (`ObtenerTodos()`), modificar o eliminar el elemento en la lista, y finalmente sobrescribir el archivo de texto completo con la lista actualizada. No es necesario preocuparse por el rendimiento de esta operación. El objetivo de esta fase es entender cómo interactúa el Caso de Uso con el contrato (`IExpedienteRepository`). En la siguiente fase, estos repositorios transitorios serán reemplazados por una implementación real con base de datos (SQLite) que resolverá estas operaciones de forma eficiente.

- **Importante:** Al leer del archivo, el repositorio debe utilizar el método `Reconstruir` (Factory Method) de las Entidades, nunca el constructor de creación normal. Si no encuentra una entidad para modificar/eliminar, debe lanzar una `RepositorioException`.
- Implementar `AutorizacionProvisionalService` (hereda de `IAutorizacionService`). Para esta entrega, su método `PoseeElPermiso` simplemente devolverá `true` siempre.
- *Nota:* Utilizaremos este servicio provisional temporalmente para no trabar el desarrollo. En la próxima entrega será reemplazado por un servicio definitivo que consultará la base de datos para verificar si el usuario realmente cuenta con los permisos solicitados.

5. Capa de Presentación (SGE.Consola)

El proyecto ejecutable actuará como nuestra interfaz de usuario y como **Composition Root**.

- En el `Program.cs`, se deberán instanciar las implementaciones de los repositorios y servicios (Infraestructura) y pasárselos por constructor a los Casos de Uso (Aplicación).
- Simular un flujo de ejecución interactuando con Casos de Uso mediante la creación de DTOs.
- Se debe probar el "Camino Feliz" (crear expedientes, agregarles trámites que cambien su estado, listar) y los "Caminos de Error" (intentar crear caratulas vacías para ver cómo actúa el Dominio, o cambiar temporalmente el retorno del `AutorizacionProvisionalService` a `false` para corroborar que salta la `AutorizacionException`).
- Utilizar bloques `try/catch` para capturar y mostrar mensajes amigables al usuario diferenciando `DominioException`, `AutorizacionException` y errores generales.



NOTA AVANZADA DE DISEÑO: El "Tiempo" en DDD (Opcional) En un modelo DDD purista, las Entidades nunca deben acceder directamente al reloj del sistema (por ejemplo, llamando a `DateTime.Now`). El reloj de la computadora se considera un detalle que pertenece a la capa de **Infraestructura**. Además, generar la fecha internamente dificulta muchísimo la creación de Tests Unitarios, ya que el valor temporal cambiará en cada ejecución de la prueba, haciendo casi imposible verificar el estado final con precisión (un `Assert`).

La solución ideal: La fecha y hora exactas de creación o modificación deberían llegar a la Entidad por parámetro en cada método (ej: `ModificarCaratula(..., DateTime fechaModificacion)`). Generalmente, esta fecha es obtenida en la capa de Aplicación a través de un servicio de infraestructura inyectado (como un `ITimeProvider`) y luego "pasada hacia abajo" al Dominio.

Para este Trabajo Práctico: Con el objetivo de no complicar el enunciado sumando más abstracciones, **hemos flexibilizado este requerimiento**. Es totalmente válido y aceptado que utilicen `DateTime.Now` directamente dentro de sus Entidades para actualizar las fechas de forma interna. Sin embargo, si alguno desea implementar la solución purista pasando las fechas por parámetro a los métodos del Dominio, ¡es la mejor manera de hacerlo y los animamos a intentarlo!

APÉNDICE A: Colecciones en Interfaces (List<T> vs IEnumerable<T>)

Cuando diseñamos los contratos (interfaces) de nuestros repositorios en la Capa de Aplicación, usar `List<T>` como valor de retorno es lo más intuitivo. Sin embargo, en arquitecturas profesionales, utilizar una interfaz más restrictiva como `IEnumerable<T>` trae enormes beneficios de diseño. Cabe aclarar que `IEnumerable<T>` es una interfaz fundamental en la plataforma .NET y la veremos en detalle más adelante en el curso. Aún así, animamos a utilizarla en los métodos de los repositorios que devuelven una lista de entidades.

Las tres razones principales para aplicar esta práctica son:

1. Memoria y Ejecución Diferida (Deferred Execution)

Un `List<T>` es una estructura de datos materializada 100% en la memoria RAM. Si en el futuro la base de datos tiene 100.000 trámites, devolver un `List<T>` obligará al servidor a reservar memoria para 100.000 objetos de golpe, saturando el sistema.

- **La ventaja de `IEnumerable<T>`:** Esta interfaz no representa "una lista cargada en memoria", sino una **promesa de entregar elementos de a uno**. Cuando nos conectemos a SQLite mediante un ORM (como Entity Framework), devolver un `IEnumerable<T>` permitirá que los datos viajen desde la base de datos a medida que el Caso de Uso los recorre en un `foreach` (como si fuera un *stream* o flujo de agua), consumiendo poquísima RAM.

2. Encapsulamiento y Protección de Datos (La "Falsa Mutabilidad")

Si la interfaz del repositorio devuelve un `List<Expediente>`, esa clase trae incorporados métodos públicos peligrosos como `.Add()`, `.Remove()` o `.Clear()`. ¿Qué pasa si un programador en la Capa de Aplicación intenta lo siguiente?

```
var expedientes = _repositorio.ObtenerTodos();  
expedientes.Add(nuevoExpediente); // ¡Falsa mutabilidad!
```

Ese código modificará la lista en la memoria RAM temporal, pero **no** guardará los cambios en el TXT o Base de Datos (porque nunca llamó a `_expedienteRepository.Agregar()`).

- **La solución:** Al devolver `IEnumerable<T>`, el compilador ocultará los métodos de escritura. Le resultará imposible al programador hacer un `.Add()` accidental, obligándolo a utilizar las vías correctas del repositorio.

3. Flexibilidad para la Infraestructura (Principio de Inversión de Dependencias)

Si la interfaz exige devolver un `List<T>`, estamos obligando a la capa de Infraestructura a instanciar

explícitamente un objeto `new List<T>()`.

- Como prácticamente todas las colecciones de C# (Arrays, Lists, HashSets) implementan por defecto `IEnumerable<T>`, usar esta interfaz en el contrato le da **libertad total al Repositorio** para trabajar con la estructura de datos que le resulte más rápida o eficiente internamente, sin romper el contrato de la Aplicación.

Devolución de `IEnumerable<T>` a partir de `List<T>`

Si un método declara `IEnumerable<T>` como tipo de retorno, pero internamente se utiliza un `List<T>`, puede retornarse la instancia de `List<T>` sin conversión adicional. Esto es posible porque `List<T>` implementa `IEnumerable<T>`. El compilador resuelve la conversión implícitamente.

```
public IEnumerable<Expediente> ObtenerTodos()
{
    // 1. Creo una lista normal
    List<Expediente> miLista = new List<Expediente>();

    // 2. La lleno leyendo del archivo TXT...

    // 3. ¡Simplemente retorno la lista!
    // El compilador la convertirá implícitamente a IEnumerable hacia afuera.
    return miLista;
}
```